

執行摘要

目前的 RAID 原型只有「條帶化」(類 RAID0) 的架構，並具備 superblock、journal 與 RAM 快取等基本模組，但缺乏真正的冗餘容錯能力。簡而言之，它目前相當於 Windows Storage Spaces 的「simple space」：雖可串接多顆磁碟為一個邏輯磁碟，卻不具備任何冗餘【1+L78-L84】【24+L163-L168】。換言之，任何單一磁碟故障都會導致整個陣列資料遺失【24+L163-L168】。此外，實作上多個子系統存在錯誤或不完整的處理：- **Mount/Unmount**：WinFsp/FUSE 參數傳遞錯誤，可能無法順利掛載磁碟。- **清除與釋放**：Cache/Flush 執行緒只等待固定 3 秒逾時就釋放，可能造成 race condition 或 use-after-free，嚴重時導致程式當機。- **Journal 恢復**：

`journal_recover_all()` 只檢查並清空 journal 卻不做完整 replay/rollback，恢復機制名存實亡。- **Superblock / Config 依賴性**：目前 superblock 與 config 的磁碟識別依賴「掃描順序」或「磁碟代號」，一旦系統重啟後磁碟順序改變，易導致錯配。- **資源管理**：部分指令在錯誤路徑未正確釋放已分配資源，例如開啟多個檔案或 handle 後部分失敗時未關閉，可能造成資源洩漏。- **錯誤回傳**：I/O 操作失敗時往往回傳 0，而非錯誤碼，導致上層誤以為寫入正常，隱匿真正的寫入失敗。- **功能不完整**：缺少 degraded 模式、重建重建、hot spare、健康檢查 (scrub) 等企業等級儲存常見功能。

`journal_recover_all()` 只檢查並清空 journal 卻不做完整 replay/rollback，恢復機制名存實亡。-

Superblock / Config 依賴性：目前 superblock 與 config 的磁碟識別依賴「掃描順序」或「磁碟代號」，一旦系統重啟後磁碟順序改變，易導致錯配。- **資源管理**：部分指令在錯誤路徑未正確釋放已分配資源，例如開啟多個檔案或 handle 後部分失敗時未關閉，可能造成資源洩漏。- **錯誤回傳**：I/O 操作失敗時往往回傳 0，而非錯誤碼，導致上層誤以為寫入正常，隱匿真正的寫入失敗。- **功能不完整**：缺少 degraded 模式、重建重建、hot spare、健康檢查 (scrub) 等企業等級儲存常見功能。

由於以上問題，這個系統當前**風險等級偏高**：在任何異常情況（例如磁碟故障、寫入錯誤、程式異常結束）下，都可能導致資料不一致或遺失。對標準 RAID/儲存系統設計而言，它尚未達到生產可靠性；**對資料安全與一致性務必謹慎對待**。以下會依優先級列出問題與修正建議，並說明每項修改步驟、測試方案與設計改善。

觀察到的問題

1. **FUSE 掛載參數錯誤 (fuse_mount_volume, fuse_bridge.c)** - `fuse_argv` 實際有 8 個參數 (含程式名) 而 `args argc` 設為 9/7 都不對，導致最後兩個 `-o` 選項被忽略。
2. 原因：`fuse_args` 的參數數目計算錯誤。實際應為參數個數 (不計 NULL) 或使用 `FUSE_ARGS_INIT` 的 macro。
3. 影響：掛載時設定未完整，如檔名 (volname)、檔案系統名未正確帶入；可能無法正確顯示並管理。無法正確掛載亦會使系統無法使用。(P0, 功能失效/掛載失敗)
4. 建議修正：校正 `args argc` 為 9，或直接用 `FUSE_ARGS_INIT(argc, argv)` 並保證 `argv[argc]=NULL`；確認所有 `"-o", "opt"` 都傳入。
5. **清除/釋放階段 race 條件 (cleanup_cache, cleanup.c)** - 目前只有在 cache thread 等待 3 秒後即強制關閉執行緒 handle。若清除逾時且執行緒尚未結束，程式會繼續釋放資源而不等待真正結束，造成 race condition 或 UAF。
6. 原因：使用 `WaitForSingleObject(thread, 3000)` 限時等待，但逾時後仍關閉 handle，並未真正讓執行緒結束。

7. 影響：若執行緒仍操作共享結構，例如 global volume，在釋放後繼續訪問已釋放記憶體，會造成崩潰或資料毀損。(P0，程序當機/記憶體問題)
8. 建議修正：改用無限等待(或設計適當退出機制)。例如只在 flush thread 以 INFINITE 等待確保完成再 CloseHandle；或向執行緒傳遞退出信號後真正 Join / Wait。不可在逾時後直接 CloseHandle (除了已確知安全的情況)。
9. Journal 恢復不完整 (journal_recover_all, journal.c) – 此函式僅掃描並分析 journal，對發現的 incomplete 事務做一次寫回，然後立即「清空 journal」，並且在迴圈內返回(只恢復第一個磁碟就退出)。
10. 原因：程式邏輯不全：只對第一個發現的磁碟處理並返回，忽略其它磁碟上的 journal。缺少實際 rollback (未實作取消已寫入範圍) 與 atomic 性。
11. 影響：多磁碟系統下僅第一顆磁碟的 journal 被回放，其他磁碟的待 commit 資料將遺失，造成資料不一致。清空 journal 後更無法再次恢復。(P0，資料不一致/遺失)
12. 建議修正：重寫 journal_recover_all()，遍歷所有磁碟、累計全體 journal entries，再執行 replay/rollback。Replay 時要先進行所有 JT_BEGIN/JT_DATA，再見到 JT_COMMIT 時才永久寫入；若碰到錯誤則取消(rollback)已寫入區段；最終一次性清空所有 journal。保證作業非破壞性地達到「整個事務成功」或「全然不做」。必要時引入寫前日誌 (write-ahead log) 或採用 Copy-on-Write 方法，如 ZFS 所示【12†L30-L34】 【13†L15-L23】。
13. Superblock 及 Config 依賴掃描序/磁碟代號 (superblock_read, config_save) – 現用的 superblock 掃描策略與 config 存放中，皆以 drive letter 或掃描時的陣列索引(disk_id)來標識磁碟，缺乏穩定 ID。
14. 原因：superblock_read 掃描所有固定磁碟，檢查 magic 及 generation，然後依 generation 選擇最佳候選。config_save 將當前物理盤列表的索引當作 disk_id 存檔。這些 ID 都非永續，例如若重插電後磁碟順序變了就會對不上。
15. 影響：如果系統重啟或插拔後磁碟符號改變，恢復時可能選錯磁碟，導致 volume 掛載到錯誤的實體盤，嚴重時覆蓋錯誤磁碟內容。(P0，資料覆寫/遺失)
16. 建議修正：改用每顆磁碟的唯一識別，如 Windows 磁碟序號 (Device Serial)、GUID、或自訂條碼。superblock 中加入磁碟 UUID/標籤，而非單純 drive letter；config.json 裡保存該 UUID 而非當前索引。組態讀入時依 UUID 對應實體設備。參考 Linux mdraid：其 superblock metadata 保存陣列 UUID、每個磁碟 role、狀態等【18†L121-L124】。
17. 錯誤回傳值處理不當 (raid_read/raid_write, stripe_engine.c, pool_io, journal) – 當 I/O 操作失敗或 volume 無效時，多數函式返回 0 而非錯誤碼，誤導上層以為操作成功。例如 raid_read/raid_write() 遇 vol==NULL 回 0；cache_flush_all() 中的 journal write 未檢查回傳值。
18. 原因：統一返回型態皆為 bool 或 uint32_t，但未明確設定錯誤碼或判斷 -1。FUSE 傳回 0 會被視作「讀到 0 bytes」，而非 I/O 錯誤。
19. 影響：上層應用 (如 FUSE) 無法察覺底層寫入失敗，導致資料未寫入且不知情。資料流可能斷裂或損毀。(P0，資料遺失)

20. 建議修正：統一以 `errno` 方式報錯。改 `raid_read/write()` 為返回負錯誤值（如 `-EIO`）而非 0，或使用 `fuse_reply_err(req, EIO)`。檢查所有磁碟 I/O 呼叫的返回值：`WriteFile`、`ReadFile`、`journal_begin/data/commit` 等，一旦失敗應立刻取消或停止並報錯，避免繼續執行後續步驟。
21. **資源釋放不完整** – 某些函式在錯誤分支中未完全釋放已分配資源。例如 `cmd_process_auto / do_restore` 等流程中，若部分命令失敗，未必回退先前的 `mount/init` 步驟；`cmd_restore` 造建命令串時若失敗返回，未釋放 `loaded_disks` 之類資源。
22. 原因：錯誤路徑缺少整體清理邏輯。命令連續執行時未捕捉每步的失敗，未呼叫相對應的 `cleanup`。
23. 影響：可能導致記憶體、`handle` 洩漏；在半初始化狀態停留（例如部分磁碟已開啟但未完成陣列組建），造成系統或資料不穩定。(**P1，資源洩漏/不一致**)
24. 建議修正：在每個步驟失敗時呼叫 `cleanup` 例程。例如 `cmd_mount()` 失敗要 `umount`，`cmd_create()` 失敗關閉所有 `handle`。最好改為「事務式」流程：成功才進下一步，否則回滾。
25. **缺乏完整功能 (需架構改進)** – 缺少 `degraded` 模式、快速重建、`hot-spare` 支援、自檢掃描 (`scrub`) 和健康監控儀表板。(**P2**)
26. 原因：目前僅實作 RAID0/1 的基本讀寫，未覆蓋 RAID4/5/6 等。未設計重建流程或備援。
27. 影響：無法在故障時維持服務。單一磁碟故障就整陣列癱瘓，用戶資料無容錯能力。(**P2，功能缺失**)
28. 建議修正：長期需引入至少 RAID1 (鏡像) 或 RAID5/6；實作**重建機制**和維護工具（如周邊掃描、寫意圖地圖等）【18†L121-L124】。加入熱備盤支援 (`Hot-spare`)。

修改計畫與優先順序

以下按照優先級 (P0/P1/P2/P3) 及風險排序提出修改事項：

- **P0 (嚴重錯誤，必須先解)：**

- 修正 FUSE 掛載參數錯誤 (`args argc` 設定、完整傳遞 `fuse_argv`)。【Fuse 掛載】
- 重寫 `cleanup/thread join` 邏輯，確保所有執行緒真正結束後才釋放資源。【Thread 安全退出】
- 完善 `journal` 恢復：跨磁碟整體 `replay/rollback`，避免只清空第一個磁碟。【Journal 恢復機制】
- 使用穩定磁碟識別 (UUID/GUID) 取代掃描順序，修正 `superblock/config` 誤匹配。【Superblock 改進】
- 統一錯誤回傳：對 I/O 錯誤返回錯誤碼，檢查 `WriteFile/ReadFile` 等結果，避免資料遺失。【錯誤處理】

- **P1 (重要缺陷)：**

- 完善所有路徑的資源釋放 (檔案 `handle`、記憶體)，尤其錯誤分支回滾。【資源管理】
- 增加參數檢查：如 `raid_read/write` 檢查 `vol->volume_valid`；`fuse_unmount_volume` 等應安全處理。
- 增加 `flush` 一致性：`cache_flush_all` 中所有 `journal` 操作後檢查返回，必要時重試或報錯。
- GUI/CLI 整潔：顯示更友善的錯誤訊息、狀態提示。

- **P2 (架構優化) :**

- 拆分四層架構：**裝置層**(掃描/辨識磁碟)、**metadata 層**(超級區 superblock、journal)、**I/O 映射層**(條帶邏輯、快取)、**服務/介面層**(CLI/GUI)。建立清晰的狀態機 (discovered/prepared/mounted/degraded/recovering/offline) 以管理生命週期，避免全域變數單向推進導致不一致。
- 設計系統狀態轉換：例如輸入「load config」->preparation->掛載->故障偵測->進入 degenerated/degraded 模式->自動重建等。【可製作 mermaid 圖示狀態機】
- 考慮更高級別的 Resiliency (mirror/parity/caching)，引入 checksums 機制以檢測及自動矯正錯誤【13+L15-L23】。

- **P3 (優化與美化) :**

- UI/UX 改善：改善 CLI 提示、日誌輸出；完善 GUI 操作流程。
- code style、註解及文件：補齊註解、統一命名規範、寫單元測試。

P0/P1 修改步驟與實作指引

1. 修正 FUSE 掛載參數 (fuse_bridge.c) 【P0】

- **問題：** fuse_args args 未正確計算 argc，導致 -o 選項缺失。
- **步驟：**在 fuse_mount_volume() 中，改用 FUSE_ARGS_INIT 巨集初始化或手動設定正確數量。範例修改如下：

```
struct fuse_args args = FUSE_ARGS_INIT(0, NULL);
const char* fuse_argv[] = {
-   "raidtest",
-   "-o", "volname=RAIDTEST",
-   "-o", "FileSystemName=NTFS",
-   "-o", "local",
-   "-o", "umask=000",
-   NULL
+   "raidtest",
+   "-o", "volname=RAIDTEST",
+   "-o", "FileSystemName=RAIDTEST",
+   "-o", "local",
+   "-o", "allow_other",
+   NULL
};
- args.argc = 9;
- args.argv = (char**)fuse_argv;
+ args.argc = (int)(sizeof(fuse_argv)/sizeof(fuse_argv[0])) - 1;
+ args.argv = (char**)fuse_argv;
```

- **說明：**將 argc 動態設為 sizeof-1 保證自動對齊。也增加了典型的 allow_other 等掛載選項。修正後確保所有 -o 參數都生效。【操作完成後可使用 fuse_mount -o help 驗證】。

2. Thread 安全退出 (cleanup_cache) 【P0】

- **問題：**Cache flush thread 和 volume.cache thread 使用固定 timeout，逾時後仍 CloseHandle。
- **步驟：**
 - 對於 flush thread：已是 `WaitForSingleObject(handle, INFINITE)`，保留。
 - 對於 volume.cache thread：移除 3 秒 timeout，改為無限等待：

```
``diff
• if (state->volume.cache.thread) {
• if (WaitForSingleObject(state->volume.cache.thread, 3000) == WAIT_TIMEOUT)
• LOG_WARN("Cache thread still running after timeout");
• CloseHandle(state->volume.cache.thread);
• }
• if (state->volume.cache.thread) {
• WaitForSingleObject(state->volume.cache.thread, INFINITE);
• CloseHandle(state->volume.cache.thread);
• } ``
```
- 確保在等待前已設置退出旗標(`state->volume.cache.running = 0`) 通知執行緒自然結束。
- **說明：**不再強制結束或忽略逾時，確保所有執行緒在釋放前已結束。可使用 `Event` 或 `volatile` 旗標協助等待。測試時可讓 Cache thread 無限迴圈並試驗中斷，確認程式無死鎖。

3. Journal 恢復機制 (journal.c) 【P0】

- **問題：**恢復程式只處理首個磁碟就立即返回，且僅做簡易 replay。
- **步驟：**
 - 移出 `return`，遍歷所有磁碟 (for loop 外 return)。
 - 為每個磁碟記錄其 incomplete transaction：蒐集所有 `JT_DATA` (payload) 範圍。
 - 等待看到 `JT_COMMIT` 才真正寫入（已部分寫的可視同失敗需 rollback）。如果發生任何寫入錯誤，需中斷迴圈並報錯。
- 完全 replay 完後，批次清空所有磁碟上的 journal 檔案。

- **程式碼示例（核心邏輯）：** `c`

```
bool journal_recover_all(STRIPE_VOLUME* vol) {
    if (!vol) return false;
    bool global_clean = true;
    for (uint32_t di = 0; di < vol->disk_count; di++) {
        // 讀取第 di 顆磁碟的 journal 檔案
        // 解析 JT_BEGIN/JT_DATA/JT_COMMIT
        bool has_begin = false, has_commit = false;
        vector<JournalEntry> entries;
        // ... (讀入 raw buffer)
        for (...) {
            JOURNAL_ENTRY je = ...;
            if (je.entry_type == JT_BEGIN) { has_begin = true; has_commit =
false; }
```

```

        else if (je.entry_type == JT_DATA && has_begin) {
            entries.push_back(je);
        } else if (je.entry_type == JT_COMMIT && has_begin) {
            has_commit = true;
            break;
        }
    }
    if (!has_commit) {
        // 實際回放：將 entries 中所有 data 段寫入 volume
        for (auto &e : entries) {
            if (!stripe_volume_write(vol, buffer_at(e.data_off),
e.virtual_offset, e.length)) {
                LOG_ERROR("Journal replay failed on disk %u", di);
                global_clean = false;
                break;
            }
        }
        LOG_WARN("Replayed journal on disk %ls (entries=%u)", root,
entries.size());
    }
    // 清空 journal 檔案
    HANDLE h = CreateFileW(path, GENERIC_WRITE, 0, NULL, CREATE_ALWAYS,
FILE_ATTRIBUTE_NORMAL, NULL);
    if (h != INVALID_HANDLE_VALUE) CloseHandle(h);
}
return global_clean;
}

```

- **說明：**整合式恢復確保所有磁碟的未完成事務正確回放，再清空。建議參考 ZFS 的「事務性寫入」觀念：要麼全部提交，要麼全部忽略【12+L30-L34】。

4. 穩定磁碟識別 (superblock.c, config.c) 【P0】

- **問題：**依賴動態掃描序號與跳動的 drive letter，不穩定。
- **步驟：**
 - 在 `SUPERBLOCK` 結構中為每顆磁碟欄位新增**唯一 ID** (如 GUID)。可取用 Win32 API (`GetVolumeInformationW` 取得卷標或序號；或讀 disk 序號、識別碼)。
 - `superblock_write()`：將每個 `DISK_INFO` 的 ID 寫入對應欄位，而非單純 drive letter。
 - `superblock_read()`：檢測到磁碟後，讀出的 ID 與 vol 的其它參考比對，確定磁碟角色。
 - `config_save()`：儲存 `DISK_CONFIG` 中每顆磁碟 ID 而非索引。可去掉 `drive_letter` 欄位或僅作顯示。
 - `config_load()`：使用載入的 ID 去 `disk_scan_all()` 得到的列表中尋找匹配項，設定 `selected = true`。

- **說明：**此方式類似 Linux mdadm 使用的 UUID；保持超級區內容獨立於系統環境。修改後即可使重啟後磁碟順序變動也能正確組成陣列【18†L121-L124】。

5. 統一錯誤處理 (raid_read/write, stripe_engine.c) 【P0】

- **問題：**失敗時回傳 0 而非錯誤。
- **步驟：**
 - 在所有 FUSE 回呼 (如 read/write) 中，如果發現底層函式 (stripe_volume_read/write) 返回 false，則呼叫 `fuse_reply_err(req, EIO)` 或等效。
 - 在 `stripe_volume_read/write`：錯誤時 return `false`；之後在 FUSE API 包裝層檢查，並轉換為 `-EIO` 回傳給系統。
 - `cache_flush_all()` 及 journaling 函式：檢查 `journal_begin()`、`journal_data()` 等返回值，一旦失敗立即中斷並返回錯誤。
- **測試例：**刻意將底層 `WriteFile` 錯誤 (例如權限拒絕) 模擬，檢查上層 FUSE/CLI 能捕捉並報錯。
- **說明：**此改善可防止隱藏錯誤，使問題可被偵測並應對。

6. 資源回滾與釋放 【P1】

- **問題：**部分命令失敗後未徹底清理先前執行步驟。
- **步驟：**
 - 檢查 `cmd_handler.c` 中所有 `cmd_...()` 函式，特別是串接命令 (`cmd_process`, `do_restore`) 中的呼叫流程。
 - 例：`do_restore()` 串了「scan/select/init/create/cache/mount」，如果中途任何一步失敗，應呼叫 `cleanup_all()` 或相應分段清理。
 - `cmd_create()`、`cmd_expand()` 已做到開檔後失敗回滾。類似地，`cmd_mount()` 需確保失敗時關閉任何 partial mount。
- 改善 `config_save()` 及其載入過程：`config_save()` 現在覆蓋全 config，若未來增欄位需兼容。載入後應再次驗證所有磁碟都已找到，否則失敗。
- **說明：**保證每步都是「要麼完成要麼不做」。可引入「檢查點」模式或 C++ RAII 風格自動資源管理(若可用)。

架構與狀態機建議

- **四層結構：**
 - **裝置層 (Device Discovery)：**負責磁碟掃描/辨識 (DiskScanner) 及 I/O 接口 (Pool I/O)。應產生穩定的 `DISK_INFO` 實例，包括唯一 ID、容量、簇大小等。
 - **元資料層 (Metadata/Superblock)：**維護 Volume 相關資訊 (RAID 級別、條帶單元、磁碟角色、UUID/版本號等)。實作持久化 (superblock)、Check-sum，以及開機重組 (assemble) 邏輯。
 - **I/O 映射層 (Stripe/Cache)：**實作條帶算法和快取。負責把單一寫入映射為多磁碟寫入 (含鏡像/parity)，並管理 Journal/MemCache (寫前日誌、快取策略)。

- **服務層 (Service/UI)**：命令列介面 (CLI)、GUI、守護進程等。負責接收管理命令，維護全局狀態機，並調用以上各層接口。

- **狀態機 (State Machine)**：

- **Discovered**：程式啟動後只執行掃描(`disk_scan`)，列出所有物理磁碟。
- **Prepared**：使用者選擇磁碟並建立（或載入）配置 (`cmd_select` / `config_load`)，構建 Volume 結構但尚未掛載。
- **Mounted**：掛載成功 (WinFsp) 並提供讀寫服務。Volume 處於正常運行狀態。
- **Degraded**：如果偵測到某顆磁碟故障 (I/O 錯誤或手動下線)，進入降級模式，仍可讀取但寫入可能只作用於正常磁碟。需發出警告。
- **Recovering**：當插入替換磁碟或重建開始時，進行資料重建。完成後狀態轉為 Mounted/Degraded。
- **Offline**：Volume 關閉、卸載完畢，或已遭遇不可恢復錯誤。需要用戶介入再啟動流程。

以上狀態間的轉換可以用 mermaid 畫 StateDiagram，自動化此流程。例如：

```
stateDiagram-v2
    [*] --> Discovered
    Discovered --> Prepared: 選擇磁碟/載入設定
    Prepared --> Mounted: 創建並掛載 Volume
    Mounted --> Degraded: 偵測磁碟失敗
    Degraded --> Recovering: 插入熱備/觸發重建
    Recovering --> Mounted: 重建完成
    Mounted --> Offline: 卸載或錯誤停機
    Offline --> Prepared: 重新準備
```

此狀態機確保操作有序，避免未經初始化就直接 I/O。

恢復與資料保護設計

- **寫前日誌**：考慮改用「寫前日誌 (write-ahead log)」或 **copy-on-write** 方法，類似 ZFS 的事務性寫入【12†L30-L34】。這可確保磁碟上永遠只有一致的狀態：要麼完全提交成功、要麼完全回退而不留部分寫入痕跡。現有的 journal 只是輔助，長期目標應研擬更強的「檢查點」機制。
- **穩定 ID**：superblock 中為每個磁碟記錄 UUID 和版本；組態檔 (config.json) 保存同樣 ID，保證裝置重新掃描也能正確對應。引入校驗碼 (CRC32) 已具備，但可拓展為加密簽章以防竄改。
- **Degraded/Rebuild**：當啟用 Mirror/Parity 模式時，須在 I/O 路徑中加入故障檢測與重建邏輯。具體：讀寫時跳過標記故障的磁碟 (mirror 可從對拷備份讀取)，在掛載時檢查「不乾淨 (dirty)」標誌，觸發背景 rebuild。可以參考 Linux md 的做法：使用 dirty bitmaps 紀錄哪些條帶需重建【18†L127-L130】。
- **熱備盤**：可選擇提供預留磁碟作為 hot spare。一旦故障偵測，立即啟動 rebuild 到這顆熱備。
- **檢查碼/自愈**：每個寫入也可計算全域 checksum (或分段 checksum) 存於 superblock 中，讀取時驗證。不匹配則嘗試從 mirror/其他副本重建【13†L15-L23】。

併發與安全釋放

- **執行緒終止模式**：對所有背景執行緒（如 FUSE thread、cache thread、daemon）使用共同退出旗標或事件。示例代碼：

```
C
// 通知執行緒退出
g_ctx.exit_flag = true;
// 在等待時，可使用 WAIT_OBJECT_0 檢查
WaitForSingleObject(g_fuse_thread_handle, INFINITE);
```

- **CancelIoEx 使用**：若有必要強制取消 I/O，應使用 `CancelIoEx(handle, &ov)` 後再 `CloseHandle`；確保未造成 incomplete I/O。

- **Flush 與 Join 範例**：

```
C
// 安全等待 cache thread 停止
SetEvent(g_ctx.cache_stop_event);
if (WaitForSingleObject(g_ctx.cache_thread, 5000) == WAIT_TIMEOUT) {
    LOG_WARN("Cache thread did not exit in time, forcing close");
    CancelIoEx(g_ctx.cache_handle, NULL); // 強制取消待處理 I/O
    WaitForSingleObject(g_ctx.cache_thread, INFINITE);
}
CloseHandle(g_ctx.cache_thread);
```

- **互斥保護**：對共用狀態（如 `g_state`，卷元資料）使用臨界區或讀寫鎖。避免同時異步快取與主寫入路徑交錯造成競爭條件。

測試計畫

- **單元測試**：為各模組撰寫單元測試。例如：superblock 讀寫正確性（不同磁碟順序）、journal replay 正確性（模擬 incomplete 狀況下預期結果）。可用 CUnit 等。
- **整合測試**：多磁碟環境（2-4 磁碟）下測試：
- 正常資料寫入/讀取：將檔案寫入虛擬磁碟，讀回校驗（如 MD5）。
- 磁碟故障模擬：寫入一些資料後強制「故障」其中一顆（例如使用鏡像模式，實際測試 RAID0 組態下斷開或標記失敗），觀察系統是否切換到 degraded 模式或正常返回錯誤。
- **Journal 測試**：對 vol 執行寫入操作但意外終止程式（殺進程），再重啟並觸發 `journal_recover_all()`，檢查資料是否恢復且 vol 完整。
- **Config 恢復測試**：將 RAID 卸載重啟後，再用 `--auto <mounted_letter>` 或 `config_load` 恢復，確保能正確找到磁碟。
- **故障注入測試**：利用工具（如模擬 I/O 錯誤的 Faulty RAID）或手動模擬各種失敗情況，確保系統處理之健壯性。
- **效能測試**：在 Windows 環境使用第三方工具（如 `dd` 或 `DiskSpd`）於掛載的卷上執行大量讀寫，評估效能。注意比較「關閉快取」與「打開快取」的差異，驗證快取效益。
- **驗證場景**：至少提供成功與失敗案例。範例：
- 成功案例：兩磁碟 RAID1 模式，進行檔案寫入後正常讀回，然後拔除一磁碟，系統仍可讀取剩餘資料並寫入錯誤。
- 失敗案例：RAID0 模式中斷電（關閉程式），再次掛載後檢查 journal 恢復結果；若 Journal 有錯，應提示並回滾。

優先補丁列表與工作量估算

編號	項目	嚴重度	功能區	工時估計
1	FUSE mount 參數	P0	Fuse 橋接層	1 小時
2	Cleanup/thread 退出	P0	Cleanup 模組	2 小時
3	Journal 恢復	P0	Journal 模組	4-6 小時
4	穩定磁碟識別	P0	Superblock/Config	4-8 小時
5	錯誤回傳處理	P0	Stripe 引擎	3-5 小時
6	資源回滾	P1	CMD Handler	3 小時
7	增強檢查與報錯	P1	各模組	2-4 小時
8	架構與狀態機	P2	全局架構	8-12 小時
9	高級功能 (重建等)	P2	I/O/Metadata 層	10-20 小時
...	...	...	...	...

具體工時依人員熟悉度而定。整體看來，先完成前 5 項即可大幅提升穩定性和安全性 (約 2-3 人天)。

參考資料

- Microsoft Storage Spaces 文件【1†L78-L84】【1†L72-L80】：介紹「simple spaces」與各種容錯類型。
- Oracle ZFS 文件【12†L30-L34】【13†L15-L23】：闡述事務性寫入 (Transactional Semantics) 和檢查碼自愈設計。
- Linux mdraid 教程【18†L57-L60】【18†L121-L124】：展示使用 on-disk superblock 存放陣列 UUID、磁碟角色等元資料。
- RAID0 概念 (Wikipedia)【24†L163-L168】：說明 RAID0 不冗餘的特性，任一磁碟故障即全丟失的風險。

以上即為對現有 RAID 原型的分析與改進建議。施工時請從**最高風險問題開始**，逐步修正並驗證，每步驟均須測試其穩定性與相容性。